

F24-128-D-Modelosaurus Mid-Eval Report

Project Team

Ahmed Raza 21I-1152

Fatima Bilal 21I-1191

Haadiya Sajid 21I-1216

Session 2021-2024

Supervised by

Dr. Atif Jilani



Department of Software Engineering

National University of Computer and Emerging
Sciences Islamabad, Pakistan

October, 2022

Table of Contents

Chapter 1: Project Scope.....	6
Introduction.....	6
1. Problem Statement.....	6
1.1 Scope.....	6
1.1.1. Scope of the Project.....	6
1.1.2. Supplementary features:.....	7
1.1.3. Limitations:.....	7
1.2 Modules.....	7
1.2.1. No-Code Model architecture customization.....	7
1.2.2. Architecture Analyzer Assistant.....	7
1.2.3. Fine tune existing models.....	8
1.2.4. Visualization.....	8
1.2.5 Model training and testing.....	8
1.2.6. User management module.....	8
1.2.7. Payment module.....	8
1.2.8. Deployment (Supplementary module).....	9
1.2.9. Dataset Creation (Supplementary module).....	9
1.3 User Classes and Characteristics.....	9
1.3.1. Developers.....	9
1.3.2. Researchers.....	9
Chapter 2: Project Requirements.....	11
2.1 Detailed Use-cases.....	11
2.1.1. Use Case 1: User Account Management.....	11
2.1.2. Use Case 2: Fine-Tune Models.....	13
2.1.3. Use Case 3: Visualize Model.....	14
2.1.4. Use Case 4: Build Dataset.....	16
2.2. Functional Requirements.....	17
2.2.1 User Authentication and Profile Management.....	17
2.2.2 Custom Model Creation.....	17
2.2.3 Dataset Management (Supplementary feature).....	17
2.2.4. Prebuilt Model Fine-tuning.....	17
2.2.5. Model Visualization.....	18
2.2.6 Model Training.....	18
2.2.7 Model Testing.....	18
2.2.8. Model Deployment (Supplementary Feature).....	18
2.2.9. Payment Management.....	18
2.2.10. Architecture Analyzer and Recommendation.....	19

2.3 Non-Functional Requirements.....	19
2.3.1 Security.....	19
2.3.2 Maintainability.....	19
2.3.4 Usability.....	19
2.4 Domain Model.....	20
2.5 Use Case Diagram.....	21
Chapter 3: System Overview.....	22
3.1 Architectural Design.....	22
3.2. Design Models.....	23
3.2.1. Activity Diagrams.....	23
3.2.2. Data flow diagram.....	27
3.2.3. System-level Sequence Diagrams.....	28
3.3 Data Design.....	32

List of Figures

FIGURE 2.4.1. Domain Model.....	20
FIGURE 2.5.1. Use case diagram.....	21
FIGURE 3.1.1. Modelosaurus System Architecture Diagram.....	22
FIGURE 3.2.1.1. Manage Users Activity Diagram.....	23
FIGURE 3.2.1.2. Fine-tune models Activity Diagram.....	24
FIGURE 3.2.1.3. Visualize Model Activity Diagram.....	25
FIGURE 3.2.1.4. Build dataset Activity Diagram.....	26
FIGURE 3.2.2.1. Data Flow Diagram.....	27
FIGURE 3.2.3.1 System Sequence diagram of Use case 1: Manage User Profile.....	28
FIGURE 3.2.3.2 System Sequence diagram of Use case 2: Finetune models.....	29
FIGURE 3.2.3.3 System Sequence diagram of Use case 3 Visualize Model.....	30
FIGURE 3.2.3.4 System Sequence diagram of Use case 4 Manage Dataset.....	31
FIGURE 3.3.1. Modelosaurus Data Design Schema.....	32

Chapter 1: Project Scope

Introduction

The purpose of this project proposal document is to collect, analyze, and define high-level needs and features of our proposed app, Modelosaurus. This document is for developers, project managers, testers, and documentation writers.

Modelosaurus is a web based application, geared towards developers who want a no-code environment to create custom ML/DL models, which are suited for their specific needs.

1. Problem Statement

In the current software landscape, creating, training and deploying ML/DL models is a considerable challenge. Developers are increasingly opting for custom models as large-scale generic models are not refined for every use case.

Designing the model architecture requires the developer to have sufficient knowledge & experience with models. Often, developers spend long hours in trial and error, while trying to find the perfect model architecture that would give the most accuracy for their particular use case and needs. Additionally, training and testing these models requires suitable hardware resources, which not everyone has access to.

Moreover, when generating responses from a trained model, developers require clarity in understanding how exactly the models are making a decision. They have to fulfill the responsibility of ensuring that the model is processing the data using the features that they intended the model to use. A visualization and explanatory tool would fulfill this need and save hours of debugging and research.

1.1 Scope

1.1.1. Scope of the Project

Users can sign up on the platform, whereupon they will be presented with options to either create a custom model from scratch or train an existing model with their dataset. In case of selecting a custom model, the user would have two options: segmentation or classification. On the other hand existing models will include chatbots and CV models eg YOLO, etc.

Before customizing the model, the user must then provide a description of the dataset they will use to train the model, and intended use case. The user would then be able to drag and drop various items from a component library to build the model. The component library would include customizable layers, activation functions and hyperparameters. While building the model with drag and drop, the architecture analyzer assistant would consistently analyze the model and provide suggestions on how to improve it, given the context of the dataset and intended use case.

Finally, the user would provide the dataset and move on to train the model on the cloud. After this, he can download the trained model or the code. He could also test the model via a GUI interface, which would provide inference visualization via XAI techniques.

1.1.2. Supplementary features:

1. Deployment of the model would be a supplementary feature. After deploying the model, the user would be able to access it via an API call or an embeddable component.
2. Text-based dataset support for custom models.
3. Open-source component library.

1.1.3. Limitations:

1. Dataset must be provided by the user and model accuracy will depend on the dataset.
2. The custom models only support image datasets, and will only perform either segmentation or classification.

1.2 Modules

Below, we will outline the key modules for the Modelosaurus project. The features under each module are highlighted using bullets.

1.2.1. No-Code Model architecture customization

Using our given component library, users can customize the architecture of the model in a no-code interface.

- A comprehensive component library will be developed, containing various visual blocks representing layers and activation functions.
- The drag-and-drop interface will enable users to create models using the component library without writing code.
- Code generation based on the architecture crafted by the user.

1.2.2. Architecture Analyzer Assistant

LLM-based assistant which enables the developer to analyze the model without actually running it.

- Provide dynamic feedback, suggestions and alternatives on the architecture of the model being created while the user is customizing it, taking into account the context of the dataset being provided and the intended use case.
- Provide information about the layers and examples of usage.
- Chat feature to allow users to ask specific questions related to the model.

1.2.3. Fine tune existing models

Provide the option of certain pre-built models that the user can finetune on his dataset. The inference of these models would be visualizable but the model architecture would not customizable

- Provide an array of pre-built models for segmentation and classification.
- Allow the user to finetune the model on his own dataset.
- HyperParameter tuning and reporting of finetune progress.

1.2.4. Visualization

Through visualization, we will introduce transparency into how the model is making a decision. This module is focused on allowing users to understand the working of a model.

- Model visualization feature
 - ◆ Visualize the structure (layers, weights, activation functions and input/output format)
- Inference visualization:
 - ◆ Given a particular input, use XAI Techniques to show which features are being extracted by the model. E.g, identify what parts of an image a model is taking into account while classifying it.

1.2.5 Model training and testing

Model training is a significant challenge, but our model training and testing module aims to simplify and streamline this process using scalable cloud methodologies.

- The user can train the model on the platform without having to run it locally.
- The user does not need to have the web application open while training is ongoing.
- Users can access a user-friendly GUI to interact with the model and test it to ensure its accuracy. It will provide an interface for both textual inputs (in case of chatbots) or images (in case of CV models).

1.2.6. User management module

Being a developer on the platform, it is essential to be able to manage your models to ensure a smooth experience.

- User will be able to create and manage his profile
- User will be able to save his created models (architecture only)
- User will be able to edit created models

1.2.7. Payment module

Training and deploying models on the cloud incurs cost, depending on the model size and training requirements. Users will be required to pay for accessing these features.

- Users will have access to a pay-as-you-go model of payment.

- Users will be able to get a prediction of the expected cost before training their model.

1.2.8. Deployment (Supplementary module)

Deployment of the trained models created on Modelosaurus is an extra feature we are aiming to add. This will introduce immense efficiency and ease of use for developers who are creating their custom models.

- We aim to include the feature to deploy the trained model in a single click.
- Provide the user with an inference API endpoint to access the model.
- Provide the user with a UI component that can be embedded into their website, in the case of chatbots. This allows the user to interact with the model directly.

1.2.9. Dataset Creation (Supplementary module)

To introduce ease for developers, we will allow them to build the dataset on our platform.

- Allow the user to create a dataset with their desired name.
- Allow the user to create labels in the dataset and upload images corresponding to each label.

1.3 User Classes and Characteristics

1.3.1. Developers

This user class has the following needs:

- Want to visualize and thoroughly understand existing models they are working with.
- Need to learn how to create custom models to keep up with the rapidly changing technological landscape.
- Need to efficiently create, train, and deploy models without requiring extensive hardware resources.
- Want to simplify the trial-and-error process in model architecture design.
- Want to gain transparency into model decision-making through explainability techniques.

1.3.2. Researchers

This user class has the following needs:

- Need to conduct experiments with custom ML/DL models tailored to specific research objectives.
- Need to analyze model architectures and evaluate their performance using detailed metrics.
- Need access to pre-built models for immediate testing on research datasets, with options for fine-tuning.

- Require clear visualizations and insights into how models make decisions, especially for research validation.
- Experiment with different datasets, including both image-based and text-based data, for multi-domain research.

Chapter 2: Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Detailed Use-cases

2.1.1. Use Case 1: User Account Management

Use case 1: User account management		Level	User goal
Scope	The system under design (Modelosaurus platform)		
Primary Actors	Developer, Researcher		
Stakeholders and interests	Developer/Researcher: - Want to sign up on the platform to access services and resources provided by Modelosaurus platform. - Want to be able to maintain a profile to store their model history, etc.		
Preconditions	For signup: - User has a valid email address. For login and editing profile: - User has successfully signed up and created an account on Modelosaurus.		
Postconditions (Success Guarantee)	- User is successfully logged into his account and able to access all relevant functions on Modelosaurus. - Any changes made by the user are updated and saved to his account.		
Main Scenario	If the new user is signing up: - The user navigates to the sign-up page. - The user enters personal details such as name, email, and password and clicks the submit button. - System verifies the user details. - System displays a confirmation message. - System redirects the user to the login screen. If the user is logging in - The user navigates to the login page. - The user enters the data in the fields and clicks submit. - The system redirects the user to his user dashboard. - The user gains access to the services provided by Modelosaurus.		

	If the user is editing his profile 1. The user logs into Modelosaurus. 2. The user navigates to the profile section. 3. The user clicks on the edit profile option. 4. The user makes the required changes in his personal information. 5. The user presses confirm. 6. The system updates the user information. If the user is deleting his profile 1. The user logs into Modelosaurus. 2. The user navigates to the profile section. 3. The user clicks on the delete profile option. 4. The system displays a confirmation modal, asking the user to confirm the action. 5. The user presses confirm. 6. The system deletes the user account. 7. The system redirects the user to the login/signup page.
Extensions (Alternate Flows)	1. If the user enters invalid data while signing up or while editing the profile: a. The system displays an error message b. The system does not let the user continue with the sign up or login process 2. If the user enters incorrect data while logging in: a. The system displays an error message and prompts the user to try again 3. If the system encounters an error whilst deleting the user's account (e.g. being unable to connect to the database:) a. The system displays an error message to inform the user that the task required was not performed successfully.
Special Requirements	• System should validate inputs for email and password strength (e.g., minimum length, special characters). • Secure storage of user passwords (hashed).
Technology and Data variations list	• Input Variations: ◦ Email: System supports both corporate and personal email addresses (e.g., @company.com, @gmail.com).

	○ Password: Must be at least 8 characters, contain uppercase, lowercase, numeric, and special characters. • Output Variations: ○ Success: System displays a confirmation message. ○ Failure: System provides detailed error messages (e.g., "Email already exists").
--	---

2.1.2. Use Case 2: Fine-Tune Models

Use case 3: Fine-Tune Pre-Built Machine Learning Models		Level	User goal
Scope	The system under design (Modelosaurus platform)		
Primary Actors	Developer		
Stakeholders and interests	• Developer: Wants to fine-tune an existing pre-built model to improve performance on their specific dataset.		
Preconditions	• User is logged in and has access to pre-built models. • User has a dataset available for fine-tuning the model.		
Postconditions (Success Guarantee)	• The pre-built model is successfully fine-tuned on the user's dataset and ready for use or further evaluation. • User receives the fine-tuned model and can utilize it for inference.		
Main Scenario	1. User logs in and selects a pre-built model from the available options. 2. User provides the dataset for fine-tuning the selected model. a. User can upload the dataset as a ZIP file. b. User can import the dataset from HuggingFace by providing its URL. 3. The system processes the provided dataset and starts the fine-tuning process on the selected model. 4. Once the fine-tuning is complete, the user is presented with the fine-tuned model, along with the relevant hyperparameters for review.		

	5. Users can save or download the fine-tuned model for further use.
Extensions (Alternate Flows)	4. 2a: If there is an error during the dataset upload (e.g., incorrect format or corrupted file), the system displays an error message and prompts the user to upload a corrected version. 5. 4a: If the fine-tuning process encounters issues (e.g., insufficient resources, invalid hyperparameters), the system notifies the user with relevant error messages and suggests possible solutions or corrections.
Special Requirements	
Technology and Data variations list	• Input Variations: ○ Model types: Pre-built models such as Yolo, ResNet, DenseNet, ImageNet. ○ Fine-tuning options: User have the option to adjust hyperparameters i.e epochs, learning rate, augmentation, batchsize before starting the fine-tuning process. • Output Variations: ○ Fine-tuned model format: The system can generate output in formats compatible with TensorFlow, PyTorch. ○ Download options: The fine-tuned model can be saved for immediate download.

2.1.3. Use Case 3: Visualize Model

Use case 4: Visualize Model Layers Architecture	Level	User goal
Scope	The system under design (Modelosaurus platform)	
Primary Actors	Developer, Researcher	
Stakeholders and interests	• Developer: Wants to visualize and understand the internal architecture of machine learning models, for learning purposes, testing, or for working with a new model he is unfamiliar with.	

	• Researcher: Needs to visualize and understand the internal working of models, such as layers and activation functions, for research and analysis purposes.
Preconditions	• The user has an existing model file that is compatible. • The model is valid and contains layers that can be visualized.
Postconditions (Success Guarantee)	• The system successfully generates and displays a detailed graphical representation of the model's architecture, allowing the user to view and interact with the layers, activation functions, and relevant parameters.
Main Scenario	1. The user logs in and navigates to the model layers visualization section. 2. The user uploads the model file. 3. The system processes the model file. 4. The system generates a graphical representation of the model's layers, showing the different types of layers (e.g., convolutional layers, dense layers, etc.), along with activation functions and parameters. 5. The user clicks on specific layers to view detailed information such as weights, activation functions, input/output formats, and other parameters. 6. The user optionally exports the architecture.
Extensions (Alternate Flows)	4a. The model file is incompatible or has issues and the system returns an error.
Special Requirements	• The visualization should support interactive options like view details about weights and activation functions, and see input/output formats. • The system should be able to handle large and complex models efficiently without causing delays or crashes. • Export functionality should allow users to generate reports (e.g., PDF) that include model layer details.
Technology and Data variations list	• Input Variations: ◦ Models: Supports various types of machine learning models. • Output Variations:

- **Visualization format:** Interactive diagrams, static images, or textual layer summaries.

2.1.4. Use Case 4: Build Dataset

Use case 5: Manage Dataset		Level	User goal
Scope	The system under design (Modelosaurus platform)		
Primary Actors	Developer		
Stakeholders and interests	• Developer: Wants to be able to easily create datasets to work with ML models.		
Preconditions	• The user has created an account on the Modelosaurus platform and logged in.		
Postconditions (Success Guarantee)	• The system successfully loads the dataset and is fully equipped and prepared to finetune or train models using the dataset.		
Main Scenario	1. The user logs in and navigates to the dataset section. 2. The user selects the option to build the dataset. 3. The user enters the name of the dataset, and optionally its description. 4. The user creates a new label and enters the label name. 5. The user uploads a batch of images corresponding to that particular label. 6. The user repeats steps 4-5 until all the desired labels have been added. 7. The user reviews the dataset. 8. The user clicks "Done" 9. The system processes the dataset and stores it. 10. The user optionally exports the dataset as a ZIP file for later use.		
Extensions (Alternate Flows)	9a. If the system encounters an unexpected server issue while uploading the dataset, an error message shall be returned.		
Special Requirements	• The system should be able to handle large and complex datasets efficiently without causing delays or crashes.		

Technology and Data variations list

- **Input Variations:**
 - **Models:** Supports various types of machine learning models.
- **Output Variations:**
 - **Visualization format:** Interactive diagrams, static images, or textual layer summaries.

2.2. Functional Requirements

This section presents the functional requirements organized by system features, along with the specific functional requirements associated with each feature.

2.2.1 User Authentication and Profile Management

- FR 1) The system shall allow users to create accounts.
- FR 2) The system shall allow users to log in.
- FR 3) The system shall allow users to update their profile information.
- FR 4) The system shall be able to store a history of models created by a user associated with his profile.
- FR 5) The system shall allow the user to deactivate his account.

2.2.2 Custom Model Creation

- FR 6) The system shall provide a component library of various model components (e.g. layers and activation functions).
- FR 7) The system shall allow the user to drag and drop the components to build the architecture of the model.
- FR 8) The system shall provide suggestions to improve the model architecture.
- FR 9) The system shall provide feedback on the model architecture.

2.2.3 Dataset Management (Supplementary feature)

- FR 10) The system shall provide the option to build an image dataset.
- FR 11) The system shall allow users to add labels in his dataset.
- FR 12) The system shall allow users to remove labels in his dataset.
- FR 13) The system shall allow users to add images against labels.
- FR 14) The system shall allow users to remove images from labels.
- FR 15) The system shall provide the option to upload a dataset as a ZIP file in a specified format.
- FR 16) The system shall provide the option to integrate a dataset from HuggingFace by providing the URL.

2.2.4. Prebuilt Model Fine-tuning

- FR 17) The system shall allow the user to select the prebuilt model.

- FR 18) The system shall allow the user to adjust hyperparameters.
- FR 25-a)** The system shall allow the user to adjust the number of epochs.
 - FR 25-b)** The system shall allow the user to adjust the batch size.
 - FR 25-c)** The system shall allow the user to adjust learning rate.
 - FR 25-d)** The system shall allow the user to choose from standard data augmentation options.
- FR 19) The system shall allow the user to use a hugging face dataset.
- FR 20) The system shall allow the user to specify the column names for images.
- FR 21) The system shall allow the user to specify the column name for labels.
- FR 22) The system shall allow users to upload dataset for fine-tuning.

2.2.5. Model Visualization

- FR 23) The system shall be able to process .h5 model files.
- FR 24) The system shall be able to process .keras model files
- FR 25) The system shall provide the user with a visualization of model architecture.
- FR 26) The system shall provide the user with detailed information of each layer.
- FR 25-a)** The system shall show the layer input and output shape
 - FR 25-b)** The system shall show the layer module.
 - FR 25-c)** The system shall show the layer weights.
- FR 27) The system shall provide inference visualisation for models.

2.2.6 Model Training

- FR 28) The system shall support training of custom models once they are ready.
- FR 29) The system shall provide the user progress report on the training progress for each epoch.
- FR 30) The system shall allow the user to disconnect while the model is training.

2.2.7 Model Testing

- FR 31) The system shall provide a GUI for model testing.
- FR 32) The system shall allow the user to input data to test their model.

2.2.8. Model Deployment (Supplementary Feature)

- FR 33) The system will allow the user to automatically deploy models as Inference API endpoints.
- FR 34) The system shall provide the user with an embeddable component.

2.2.9. Payment Management

- FR 35) The system shall implement a pay-as-you-go pricing model.
- FR 36) The system shall allow users to add payment information.

2.2.10. Architecture Analyzer and Recommendation

- FR 37) The system shall allow the user to get feedback on model architecture.
- FR 38) The system shall provide the user with the option to explain the model architecture.
- FR 39) The system shall provide the user with suggestions on improvements in model architecture.

2.3 Non-Functional Requirements

2.3.1 Security

- NFR 1) The system shall store user authentication credentials in encrypted form.
- NFR 2) The system shall store the users' datasets securely.
- NFR 3) The system's API endpoints shall be secured using HTTPS and authenticated using token-based authentication to prevent unauthorized access to user data.

2.3.2 Maintainability

- NFR 4) All system components shall be thoroughly documented with clear comments and guidelines to ensure easy maintainability by developers.
- NFR 5) The system should have a modular design to allow ease in modification.

2.3.4 Usability

- NFR 6) The system shall provide an intuitive user interface (UI) for users to easily use the platform without requiring extensive training.
- NFR 7) The system shall provide easy-to-access help guides and tutorials within the platform, explaining how to use key features.
- NFR 8) The system shall provide clear and descriptive error messages whenever users encounter issues.
- NFR 9) If an error occurs, The system shall guide users through how to solve it.

2.4 Domain Model

This domain model shows the relationships between users, datasets, and machine learning models. Users manage datasets (custom or from Hugging Face), create and visualize models, and run various jobs like training, inference, and analysis. Models use datasets and can be custom-built or pre-built, with payments likely required for certain operations.

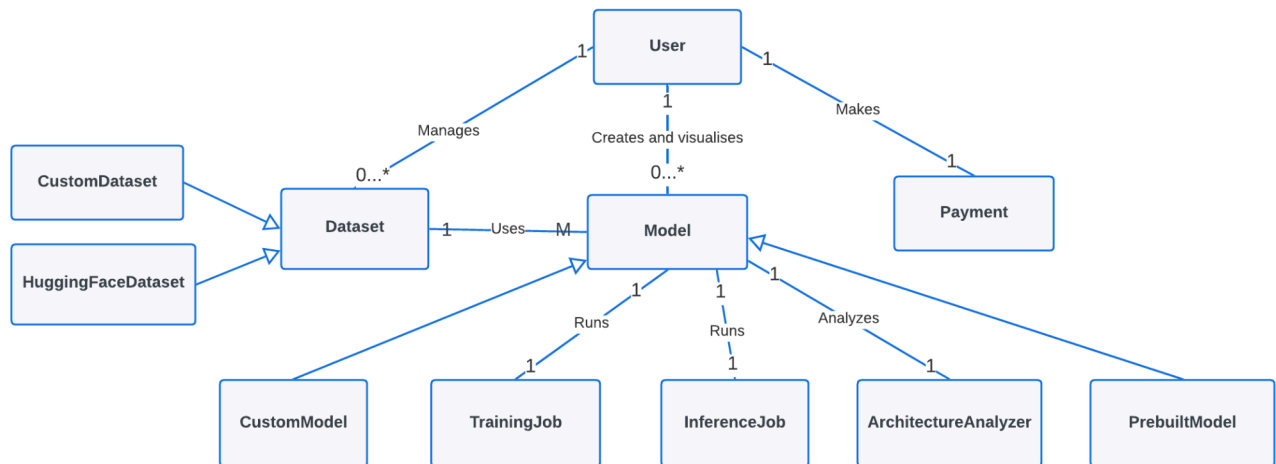


FIGURE 2.4.1. Domain Model

2.5 Use Case Diagram

As illustrated in the use case diagram, the platform offers functionalities for user management, data handling, model training, testing, and deployment.

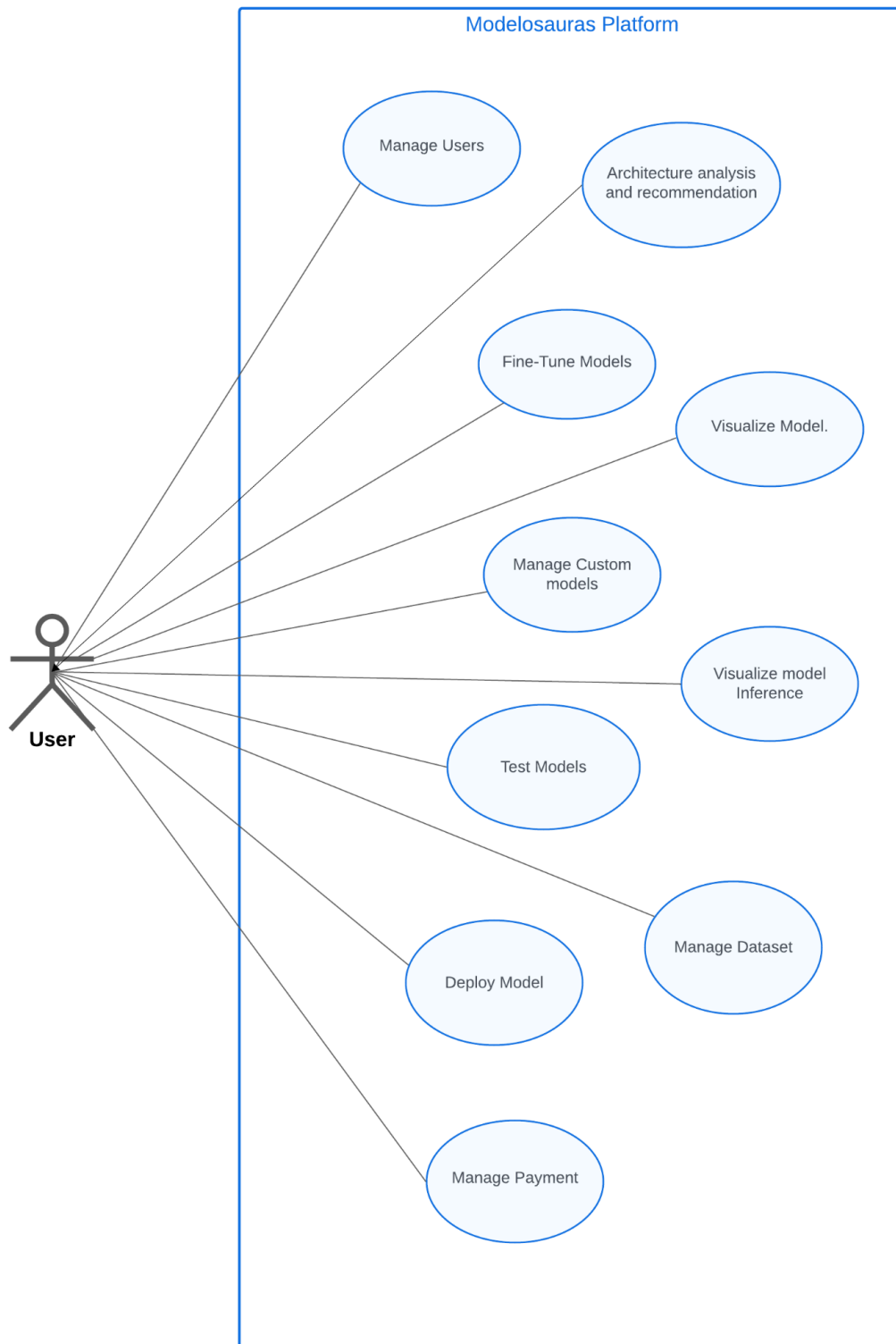


FIGURE 2.5.1. Use case diagram

Chapter 3: System Overview

3.1 Architectural Design

In figure 3.1.1., we have outlined a high-level architecture of the Modelosaurus project. The application shall follow a microservices architecture, by breaking up the project into separate, scalable microservices connected through the API gateway.

The backend houses microservices for each major feature of the project. A WebSocket enables real-time communication with the training and fine-tuning service to provide the user updates on the training progress. External integrations include Stripe for payments, HuggingFace for datasets, and Amazon SageMaker for deployment.

The architecture uses Amazon S3 buckets storage for models and a database for user management. This system is modular, supporting a wide range of machine learning workflows, from model customization to deployment.

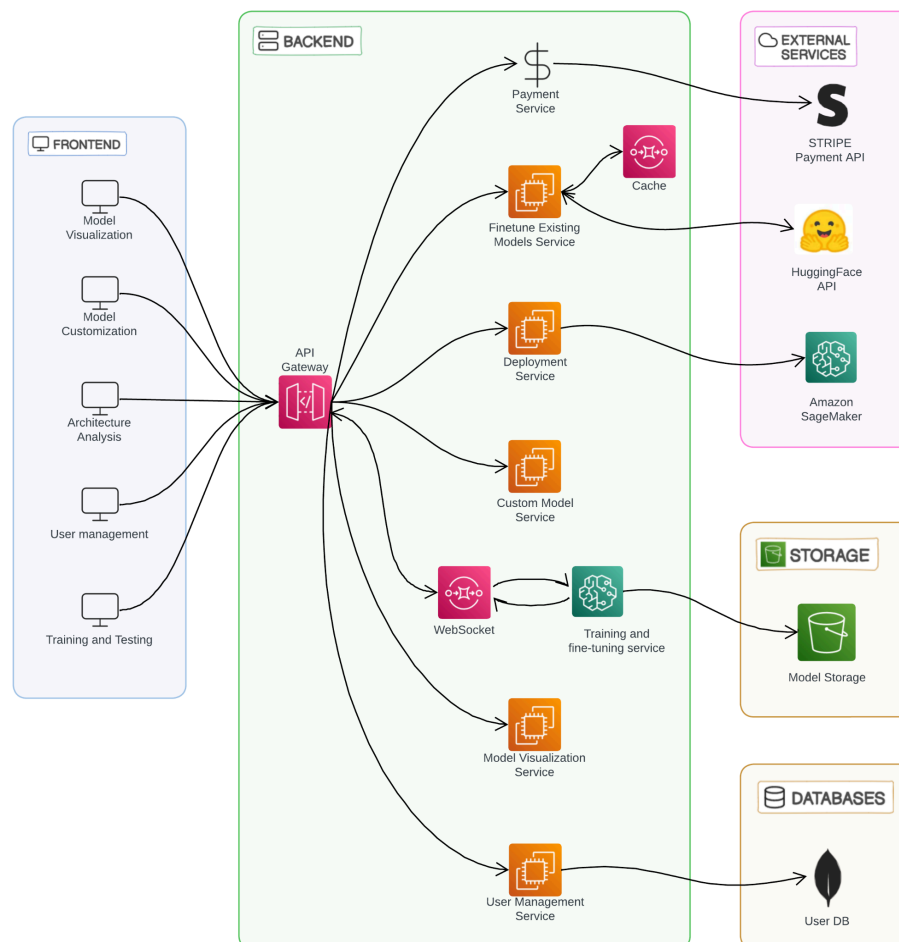


FIGURE 3.1.1. Modelosaurus System Architecture Diagram

3.2. Design Models

The design models elaborated below offer an enhanced overview of the structure and inner workings of the project.

3.2.1. Activity Diagrams

The activity diagrams corresponding to each of the use cases outlined in chapter 2 are provided below.

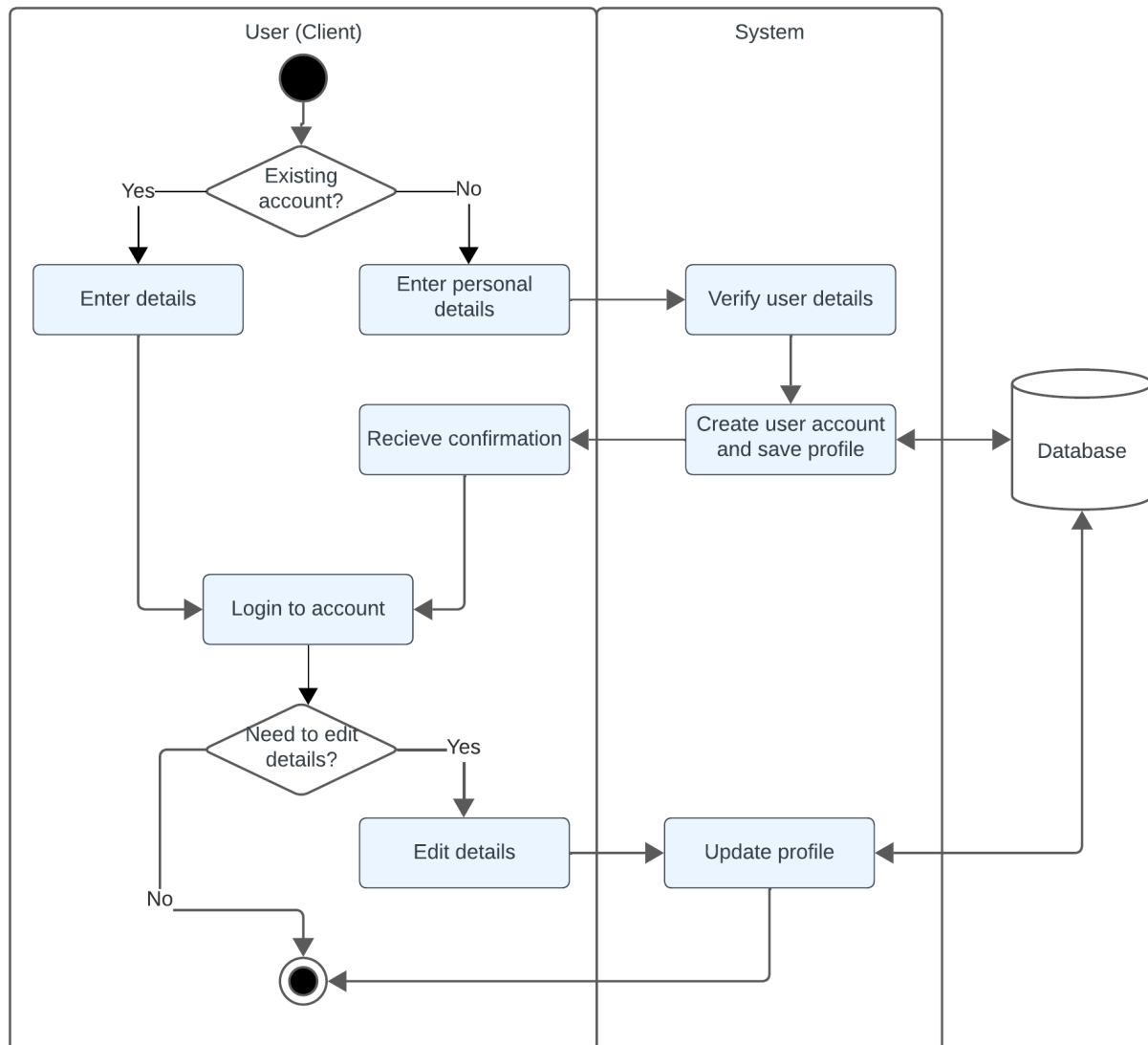


FIGURE 3.2.1.1. Manage Users Activity Diagram

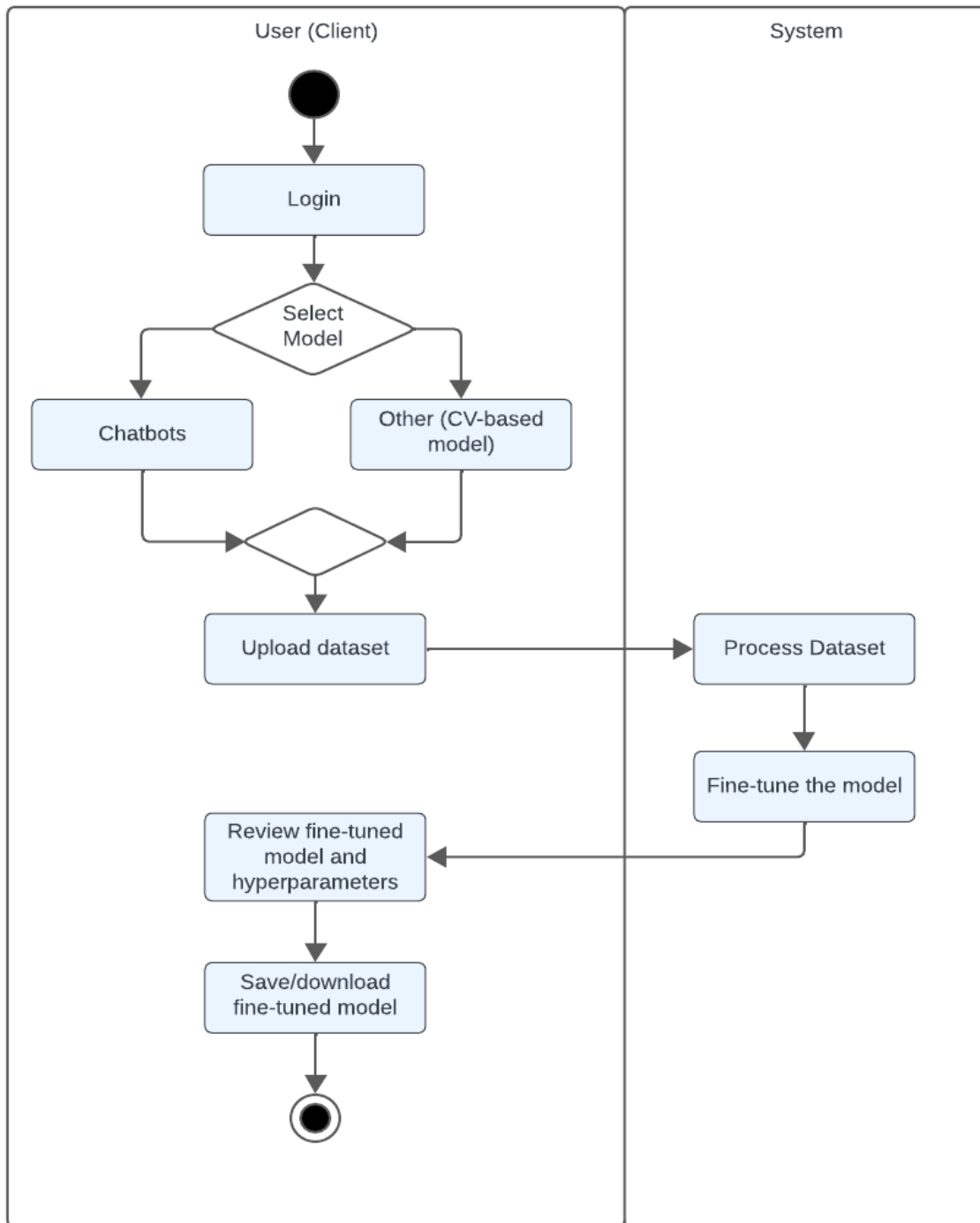


FIGURE 3.2.1.2. Fine-tune models Activity Diagram

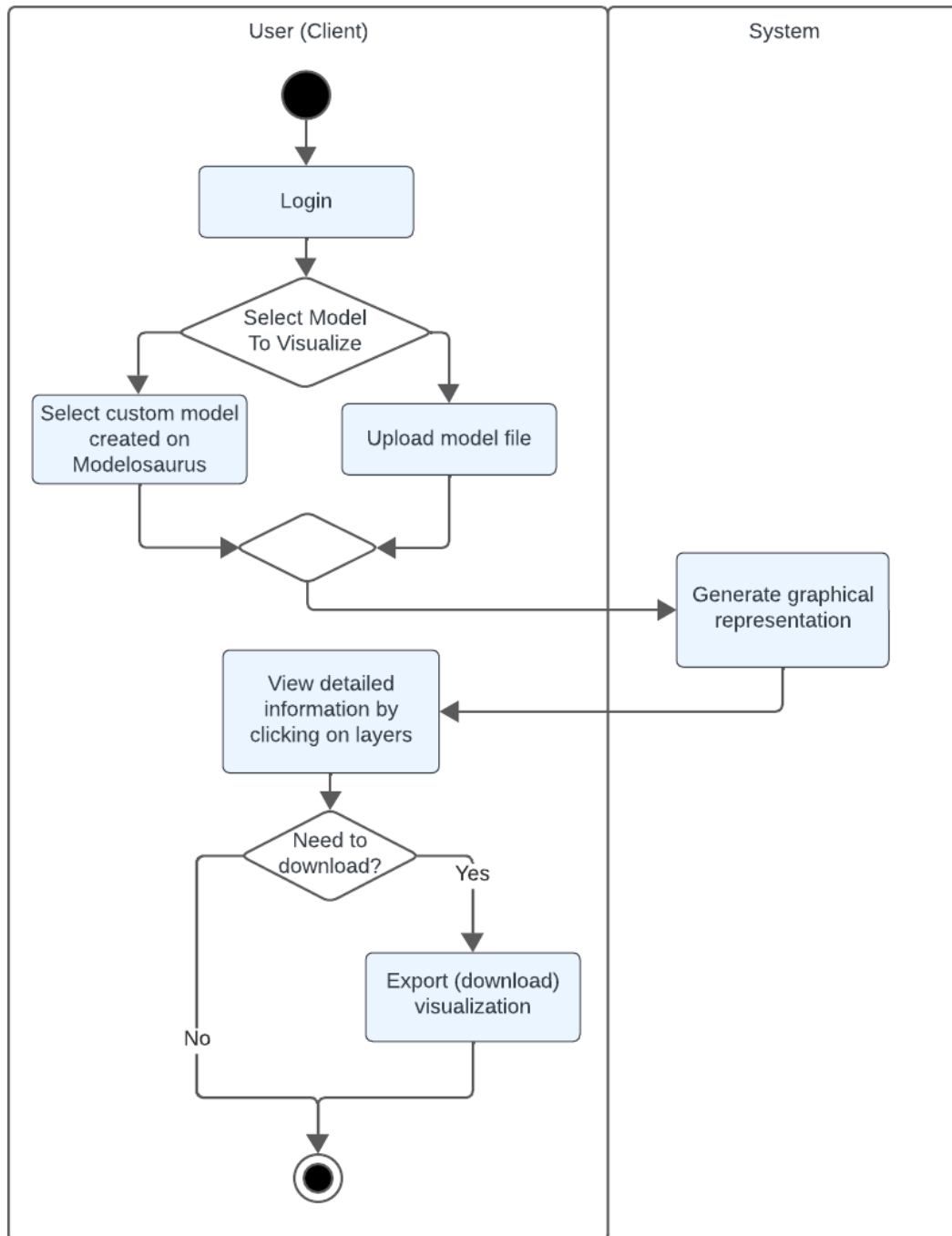


FIGURE 3.2.1.3. Visualize Model Activity Diagram

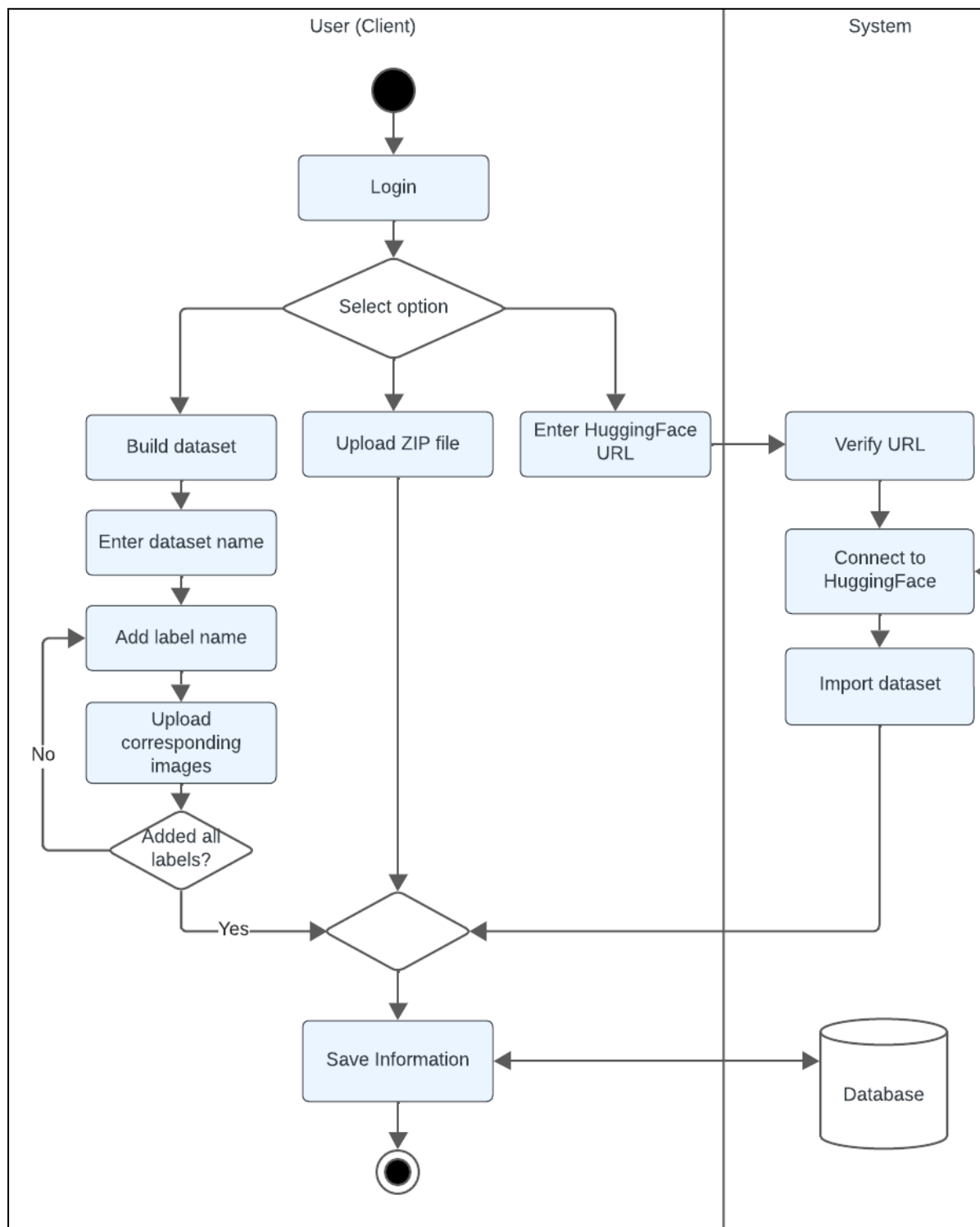


FIGURE 3.2.1.4. Build dataset Activity Diagram

3.2.2. Data flow diagram

The Modelosaurus system's data flow shows the interaction between the microservices, as follows:

1. Dataset Management handles user-provided data, linking with Hugging Face for dataset integration.
2. Model Customization takes user-defined architectures and hyperparameters, feeding them into Model Training. The training progress is tracked and stored.
3. Architecture Analysis assists users in refining their model architecture.
4. Model Deployment utilizes the trained model to deploy it. Deployed models and their corresponding data are stored in a Storage Bucket for future inference and visualization.
5. Payment Processing and User Management support the financial and user-related activities within the system.

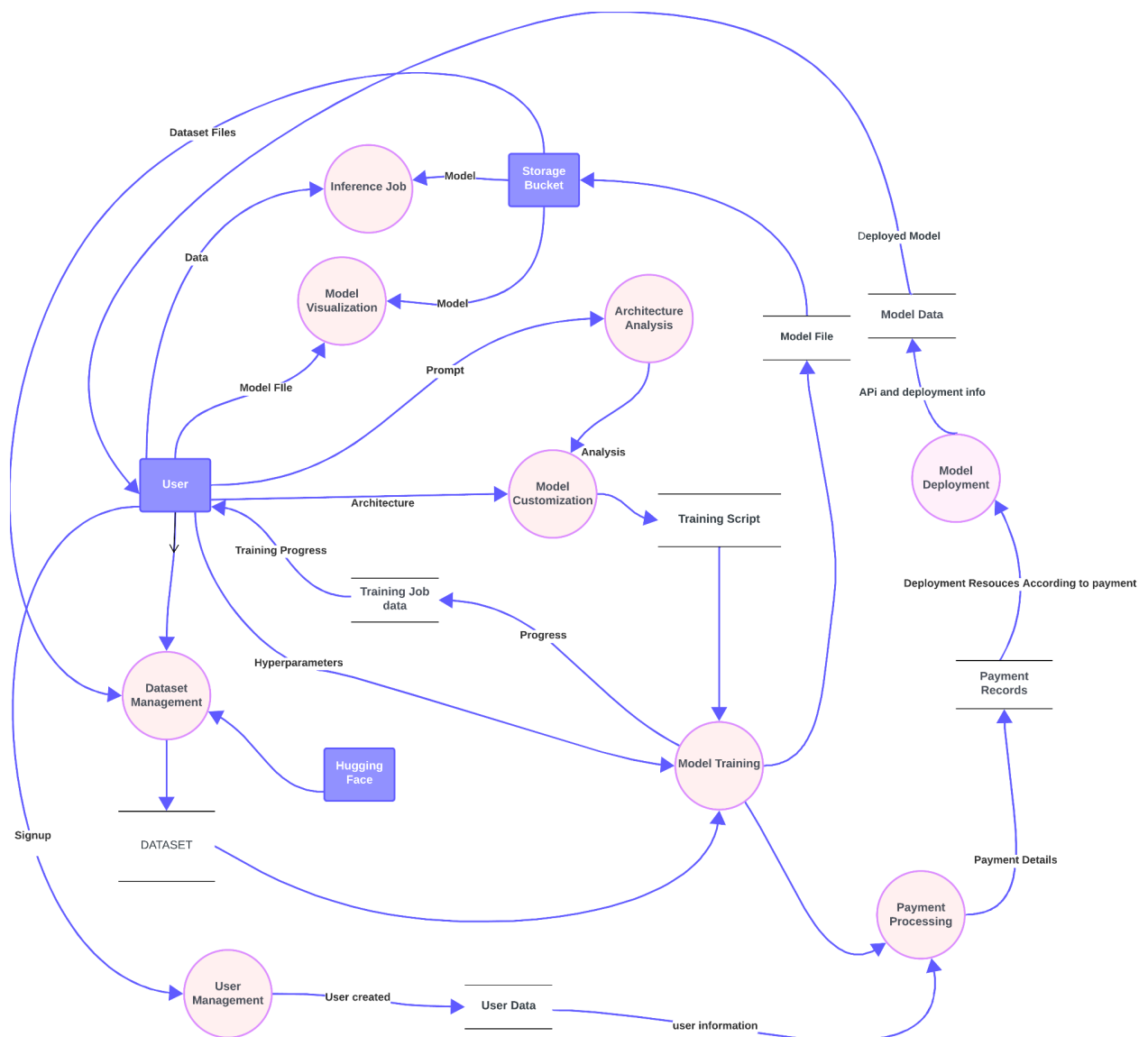


FIGURE 3.2.2.1. Data Flow Diagram

3.2.3. System-level Sequence Diagrams

Use Case 1: User Profile Management

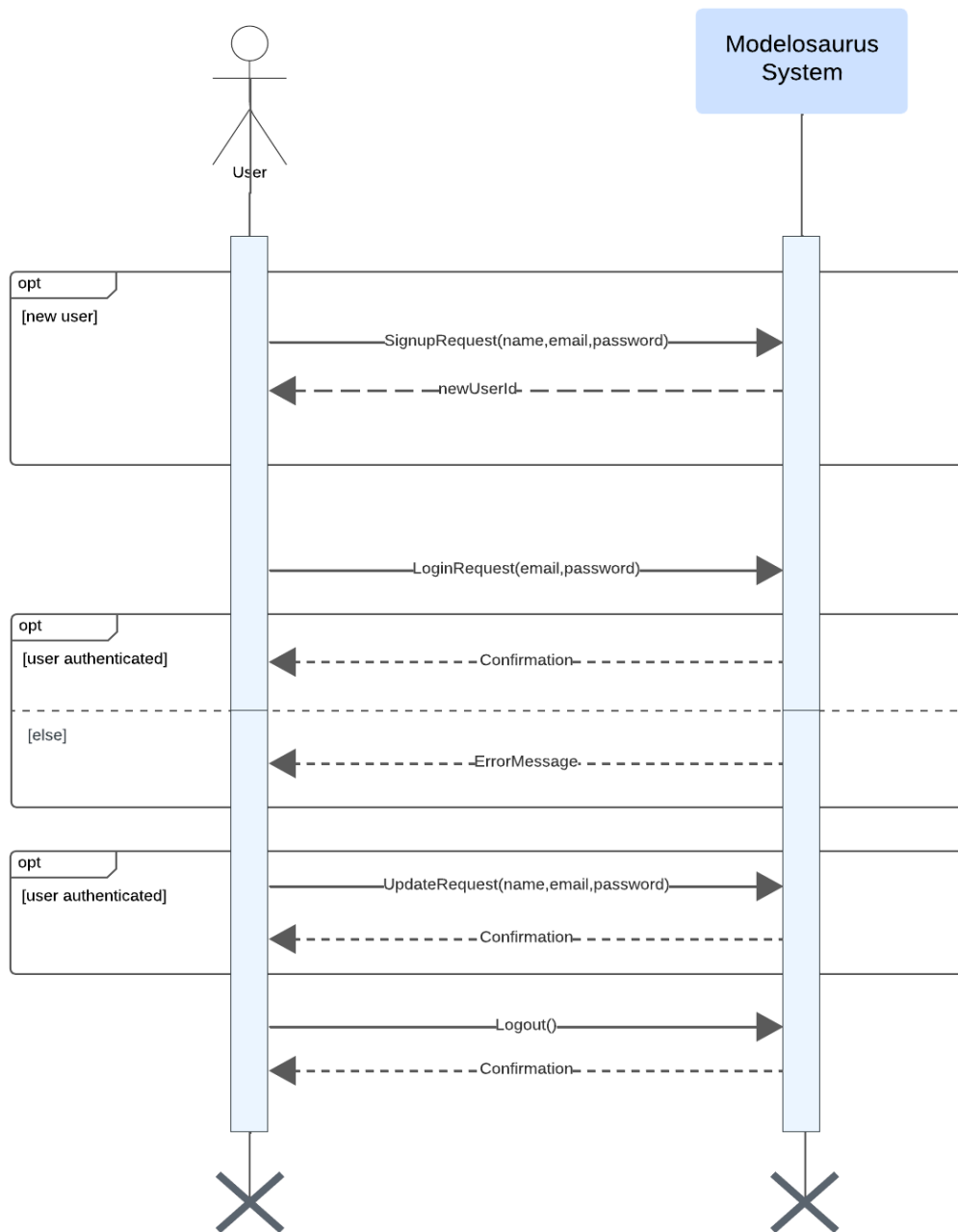


FIGURE 3.2.3.1 System Sequence diagram of Use case 1: Manage User Profile

Use Case 2: Fine-Tune Pre-Built Machine Learning Models

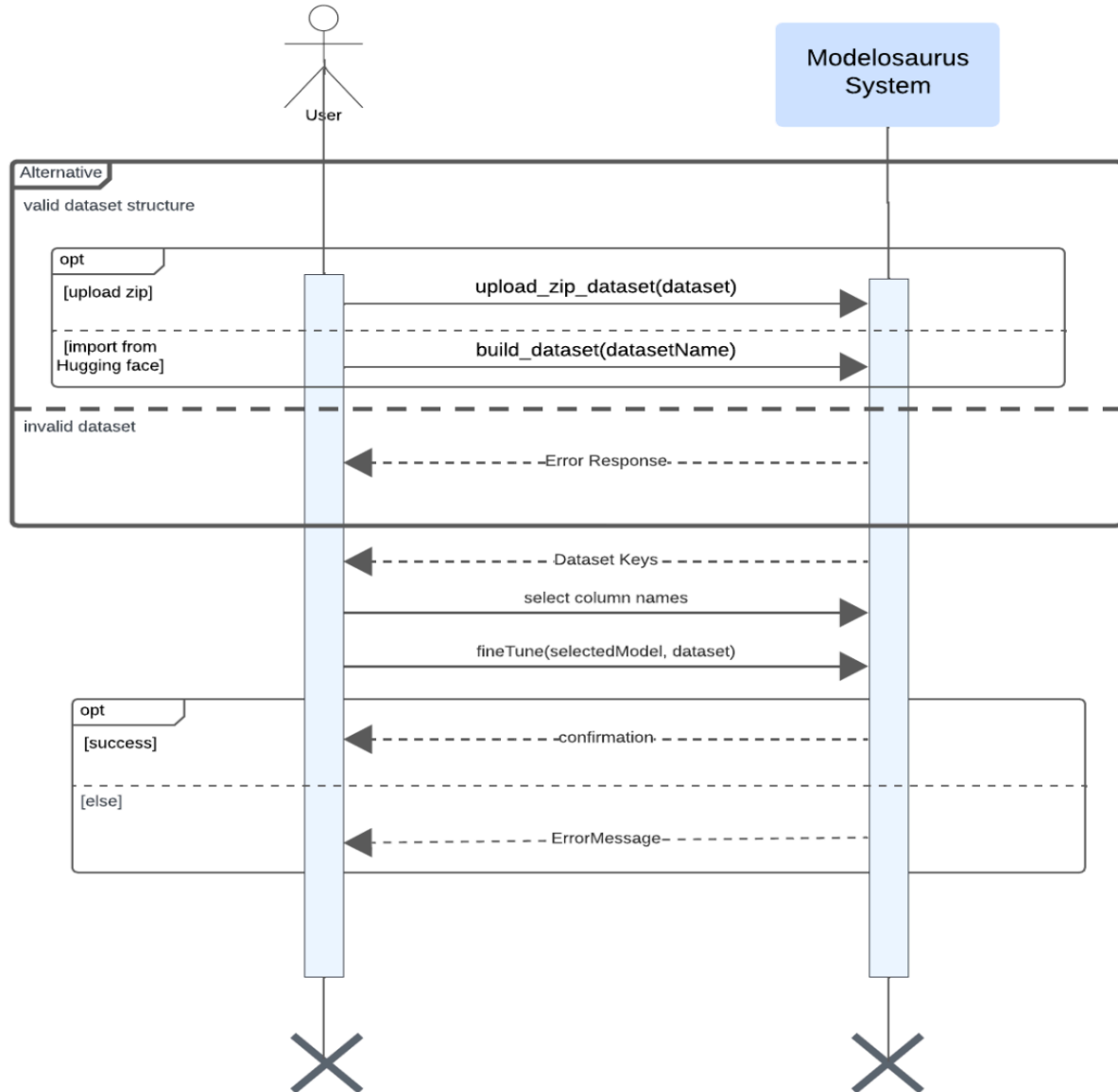


FIGURE 3.2.3.2 System Sequence diagram of Use case 2: Finetune models

Use Case 3: Visualize Model Layers Architecture

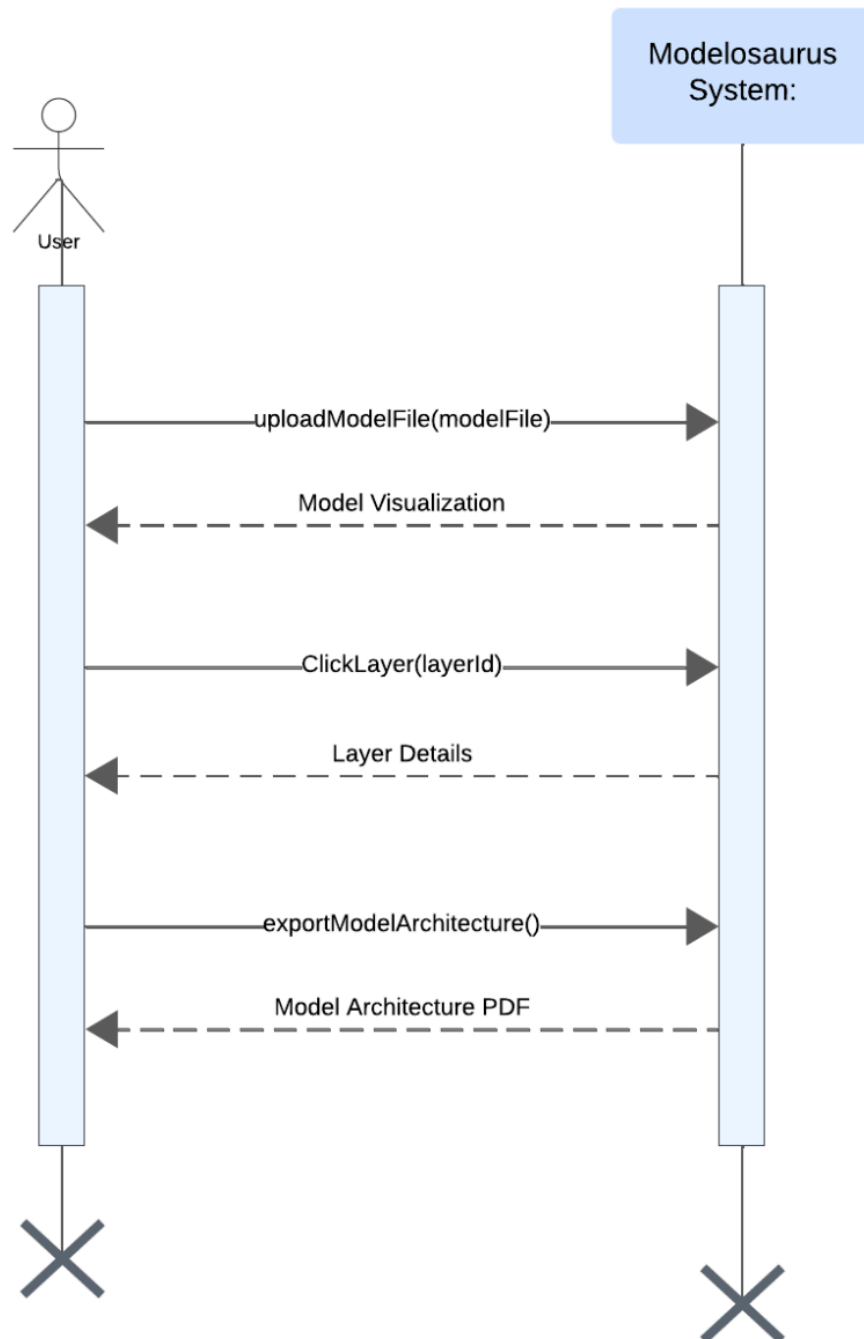


FIGURE 3.2.3.3 System Sequence diagram of Use case 3 Visualize Model

Use Case 4: Manage Dataset

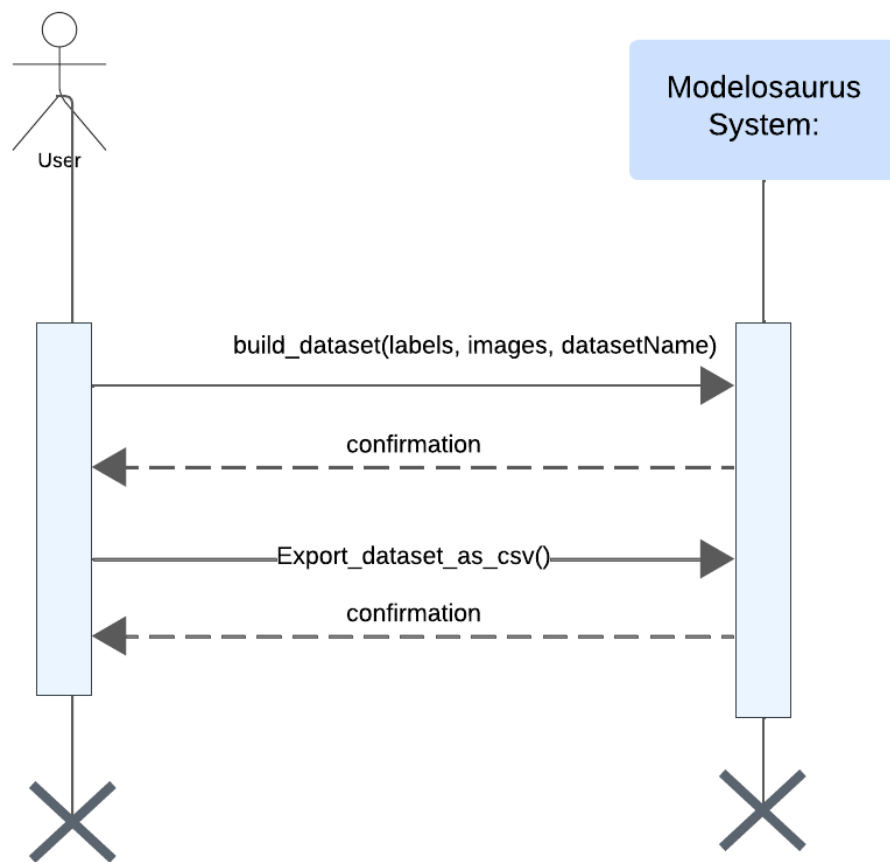


FIGURE 3.2.3.4 System Sequence diagram of Use case 4 Manage Dataset

3.3 Data Design

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed, and organized. List any databases or data storage items.

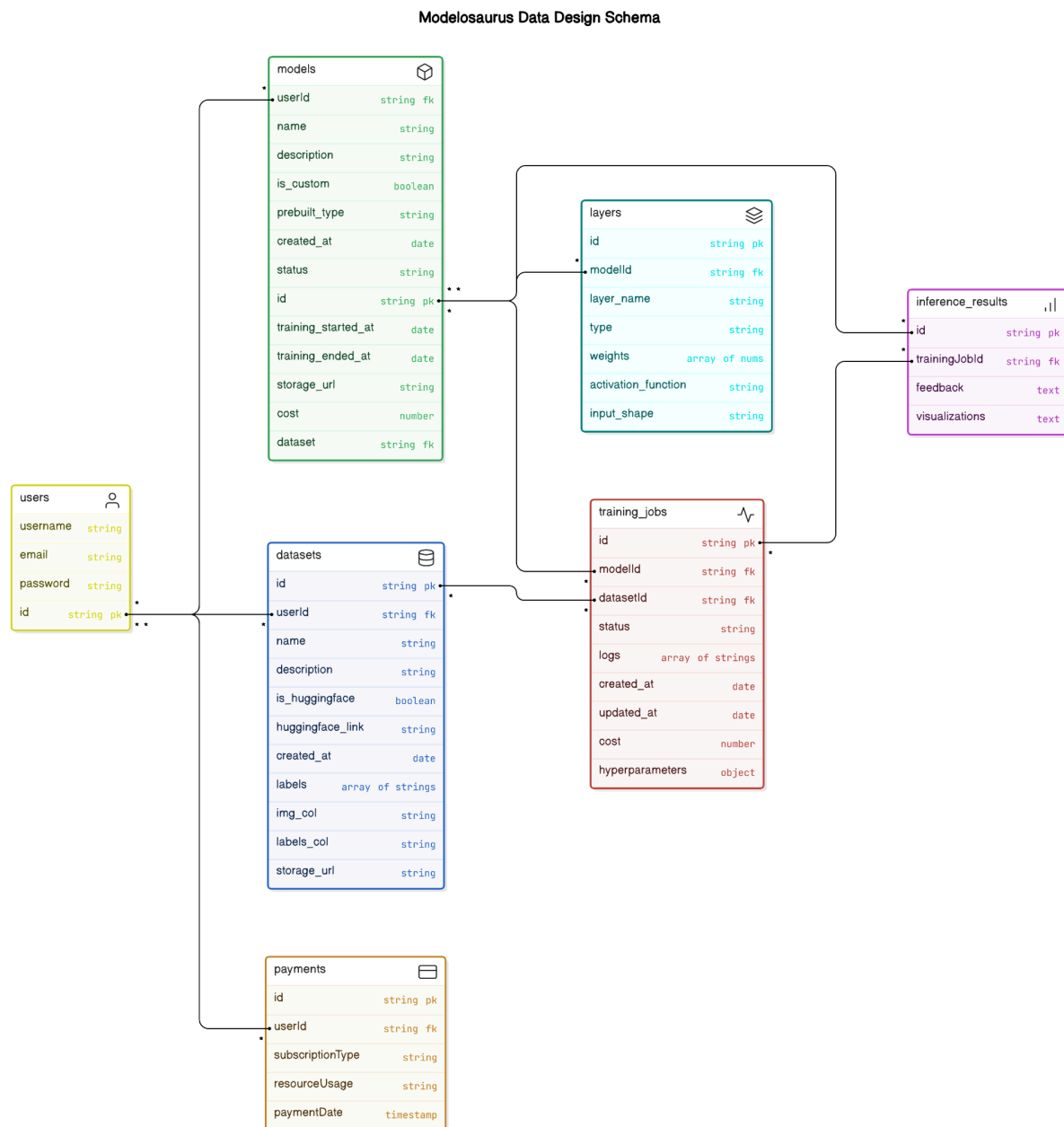


FIGURE 3.3.1. Modelosaurus Data Design Schema